

Lecture 3

An Introduction To Artificial Neural Networks

Reference(s):

- ***An Introduction to Neural Networks for Beginners***, Dr. Andy Thomas (Adventures in Machine Learning).
- **<https://www.analyticsvidhya.com/blog/2021/11/a-comprehensive-guide-to-linear-regression-with-pytorch/>**
- ***Artificial Intelligence: A Guide to Intelligence Systems***, Michael Negnevitsky, Addison Wesley
- chatGPT

Introduction to Neural Networks

- An **artificial neural network (ANN)** or **neural network (NN)** is a software implementation of the neuronal structure of a human brain.
- The brain contains **neurons** which are kind of like biological switches.
 - These can change their **output state** depending on the strength of their electrical or chemical **input**.
 - The **NN** in a person's brain is a hugely interconnected network of **neurons**, where the **output** of any given neuron may be the **input** to thousands of other neurons (massively parallel structure!).

Introduction to Neural Networks

- **Neural learning** occurs by repeatedly activating certain neural connections over others, reinforcing those connections.
- This makes them more likely to produce the desired outcome given a specified input
 - This learning involves **feedback** – when the **desired outcome** occurs, the neural connections causing that outcome become strengthened.

Introduction to Neural Networks

- **NNs** attempt to simplify and mimic brain behavior.
- They can be trained in a ***supervised*** or ***unsupervised*** learning manner.
- In a **supervised NN**, the network is trained by providing matched **input-output data samples**, with the intention of getting the ANN to provide the **desired output** for a given **input**.

Supervised NN An Example

- Consider an e-mail **spam filter** – the **input training data** could be the count of various words in the body of the email, and the **output training data** would be a classification of whether the e-mail was truly spam or not.
- If many examples of e-mails are passed through the NN allows the network to learn what input data makes it likely that an e-mail is a spam or not.
- This learning takes place by adjusting the **weights** of the NN connections.

Unsupervised NN

- In **unsupervised learning**, the NN tries to “understand” and generate the **output** structure of the provided input data set “on its own” without its output pattern.
 - There is no supervised learning happening at this point

Structure of an Artificial Neuron

- The **neurons** that we are going to see here are not biological but are **Artificial Neurons**
- The **artificial Neurons** are extremely simple abstractions of **biological neurons**, realized as elements in *a program* or perhaps as *a circuit* made of silicon.
- Networks of these **artificial neurons** do not have a fraction of the power of the human brain, but they can merely be trained to perform useful functions.

The Structure of an Artificial Neuron

- A *single-input, single-output* artificial neuron is shown in **figure 1**.
- The scalar input **N** is multiplied by the scalar weight **W** to form **$W*N$** , one of the terms that are sent to the **summer unit**:
 - Where **$N = \{n_1, n_2, n_3, \dots\}$** and **$W = \{w_1, w_2, w_3, \dots\}$**
 - The summer output is often referred to as the **net input**. It incorporates a **threshold** (or **bias**), the **weight of the +1 bias element** that goes into a **transfer function (or activation function) f** , which determines and produces the scalar neuron output **y** .

The Structure of an Artificial Neuron

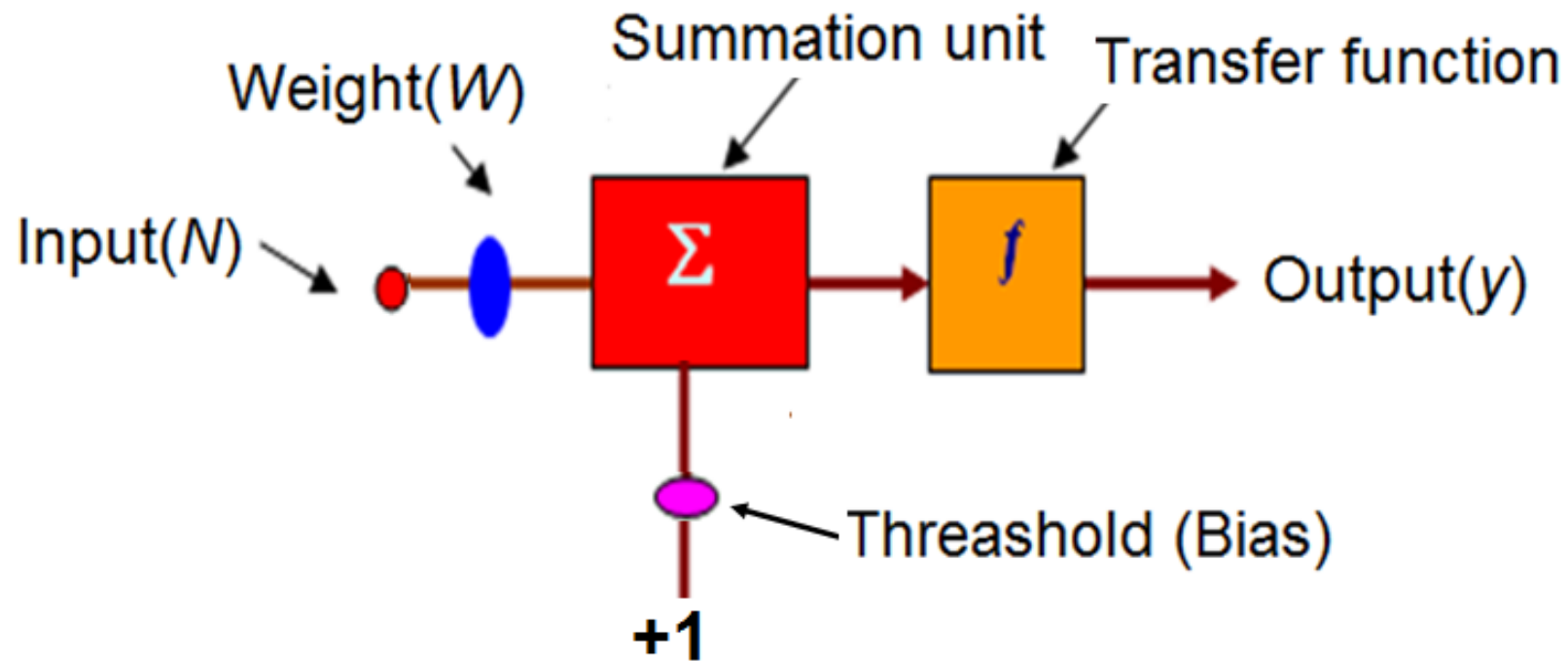


Figure 1: A single input, single output neuron

Neuron as Simple Computing Element

- An artificial **neuron** receives several signals from its **input** links, computes with its **activation function (activation level)**, and sends the result as an **output** through its **output links** (**figure 2** shows a neuron with n inputs)
- An activation function simulates the neuron in an ANN.
- The **input** signal can be *raw data* or *outputs* of other neurons
- The **output** signal can be either a *final solution* to a problem or an *input* to other neurons.

Neuron as a Simple Computing Element

- The single neuron node which is shown in **figure 2** is called as a **perceptron**.

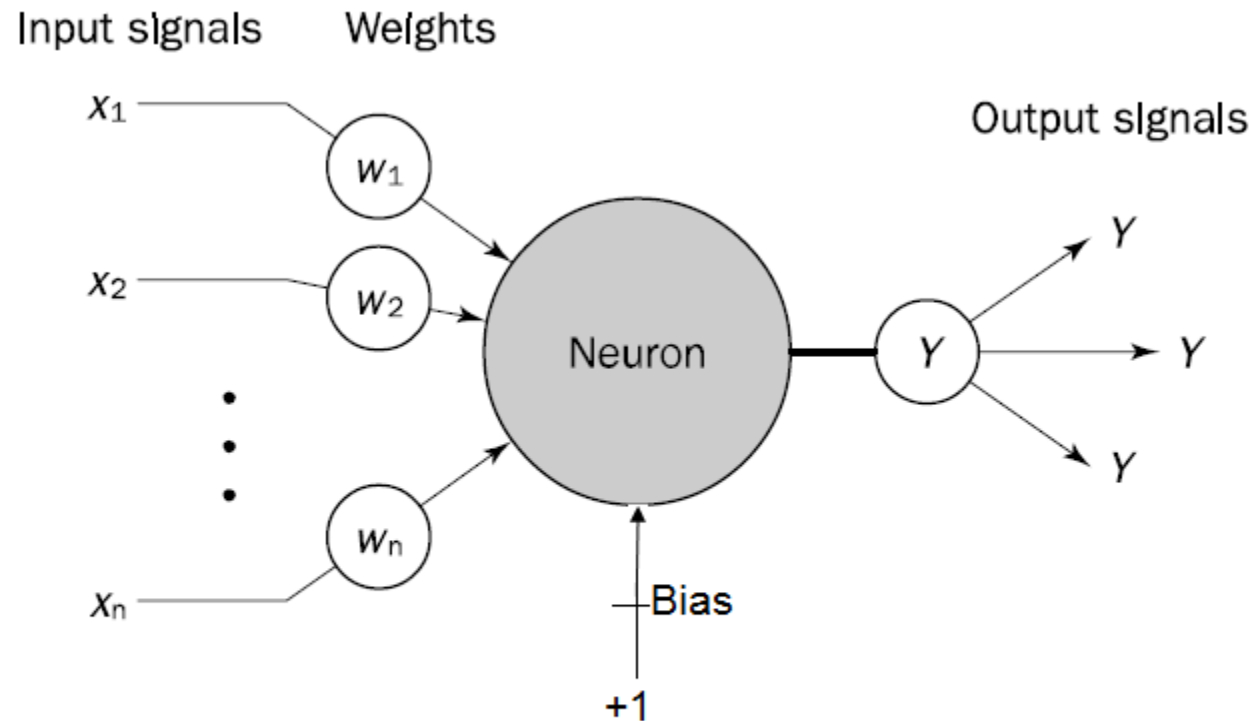


Figure 2. Single neuron with n inputs

How Does a Neuron Determine its Output?

- The neuron computes the **weighted sum** of the input signals and compares the result with a **bias** value ***b***.
- *If the **net input** is less than the ***b***, then the neuron output is **-1***
- *But if the **net input** is greater than or equal to the **bias *b***, the neuron becomes activated, and its output attains a value **+1** (McCulloch, 1943)*
- From **Figure 2**, the *sum of the product of inputs and weights*, called the **weighted_sum**, can be given as:

$$\text{weighted_sum} = \sum_{i=1}^n (x_i w_i) \quad (1)$$

$$\text{sign_output, } Y_{\text{sign}} = \begin{cases} +1 & \text{if } \text{weighted_sum} \geq b \\ -1 & \text{if } \text{weighted_sum} < b \end{cases}$$

- Where ***x_i*** is the input, ***w_i*** is the weight of the input, ***b*** is the bias, and ***n*** is the number of inputs.

How Does a Neuron Determine Its Output?

- The output of the neuron **Y** (from **Figure 2**) is estimated based on an **activation** or **output** function. Some of the activation functions are shown in **Figure 3** (*step, sign, linear, and sigmoid functions*).

The activation function shown in the previous slide is called a **sign activation** function, which indicates a **+1** or **-1** output of the neuron.

- The **weighted_sum** of the neuron can be combined with its **bias** for determining its **intermediate output (X)**, and that is used with an **activation** function to produce the **final neuron output y**.
- The **intermediate output (X)** can be given as:

$$\mathbf{X} = \sum_{i=1}^n (x_i w_i) + b \quad \text{or} \quad \sum_{i=1}^n (x_i w_i) - b \quad (2)$$

Then the output (step function), $\mathbf{Y}_{\text{step}} = \begin{cases} +1 & \text{if } X \geq 0 \\ -1 & \text{if } X < 0, \end{cases}$

where **X** is the intermediate output of a neuron.

Activation Functions of a Neuron

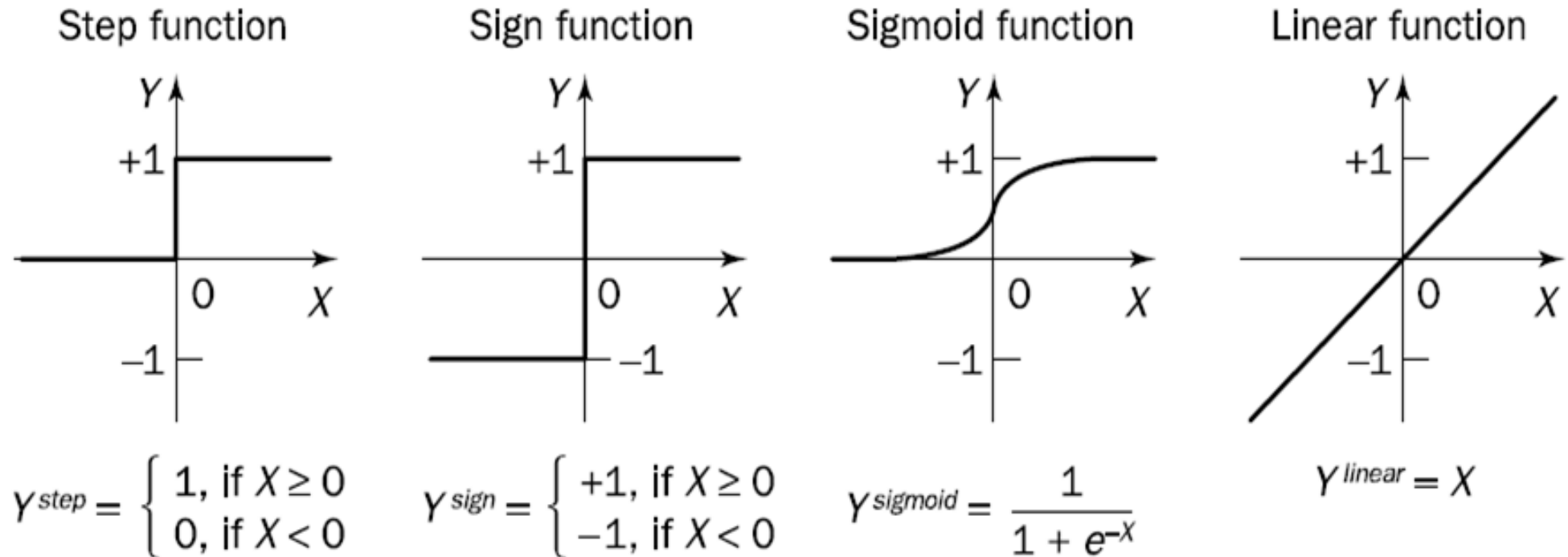


Figure 3: Activation functions of a neuron

Activation Functions of a Neuron

- The **step** and **sign** activation functions, also called **hard limit** functions, are often used in **decision-making neurons** for **classification** applications
- The **sigmoid** function ($Y_{sigmoid} = \frac{1}{1 + e^{-X}}$, where **e** = 2.7183)

transforms the **input**, which can have any value between **plus** and **minus infinity**, into a reasonable value in the range between **0** and **1**

- Neurons with this function are used in the **back-propagation** networks
- The **linear function** ($Y_{linear} = X$) provides an *output equal to the neuron's weighted input*
 - Neurons with *linear functions* are often used for *linear approximation*

The Sigmoid Function

- The **Sigmoid** activation function: ($Y_{sigmoid} = 1/(1 + e^{-x})$)

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(-8, 8, 0.1)
f = 1/(1 + np.exp(-x))
plt.plot(x, f)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.show()
```

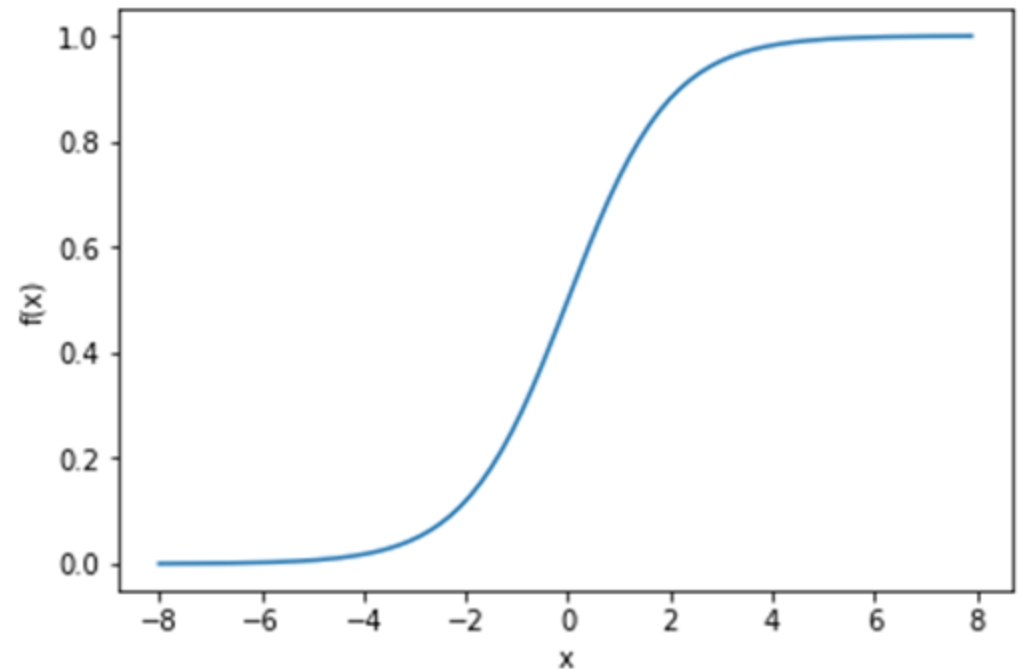


Figure 4 The sigmoid function.

The Sigmoid Function

- **Properties of Sigmoid Function**

- The sigmoid function returns a **real-valued** output.
- The first derivative of the sigmoid function will be non-negative or non-positive.
 - **Non-Negative:** If a number is greater than or equal to zero.
 - **Non-Positive:** If a number is less than or equal to Zero.

- **Sigmoid Function Usage**

- The Sigmoid function used for **binary classification** in the logistic regression model.
- While creating ANNs, uses the sigmoid function as the testing **activation function**.
- In statistics, the **sigmoid function** graphs are standard as a **cumulative distribution function**.

Activation Function: tanh

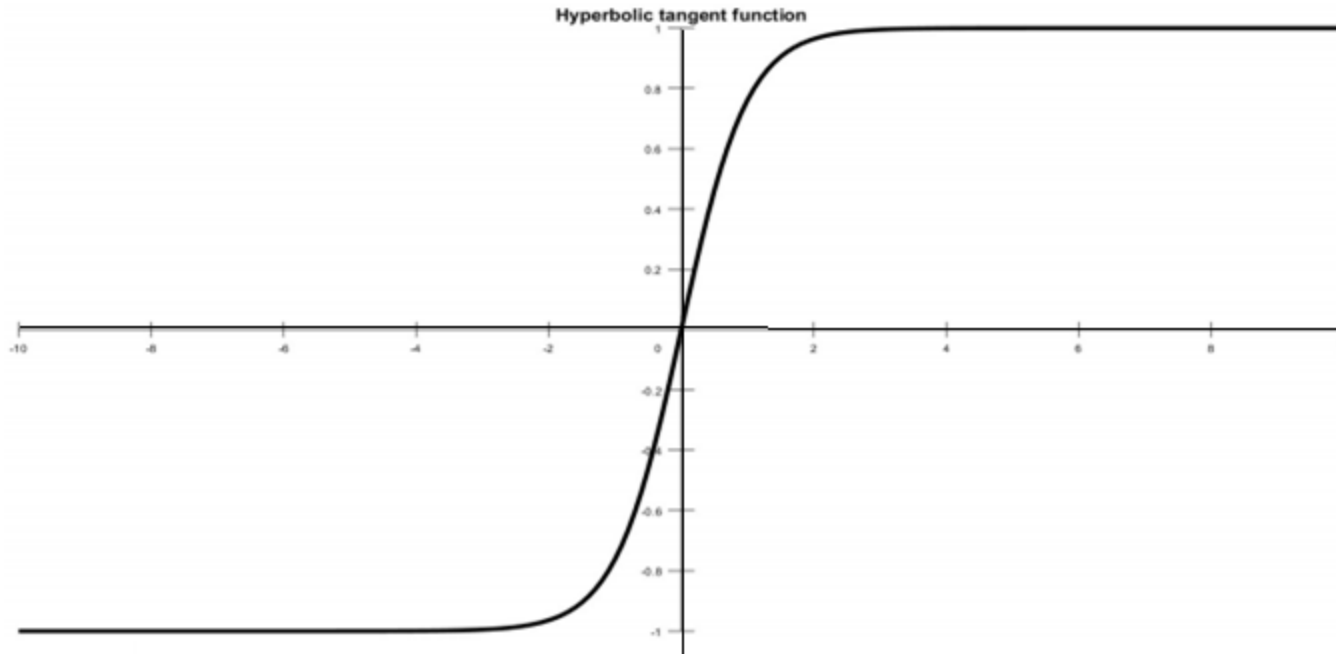
- Another popular NN activation function is the **tanh (hyperbolic tangent)** function.
- The **tanh** function is defined as:

$$f(x) = \tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

- It looks very similar to **sigmoid** function; in fact, **tanh function is a scaled sigmoid function**.
- As **sigmoid function**, this is also a **nonlinear function**, defined in the range of values **(-1, 1)**.
- The **gradient (or slope)** is stronger for **tanh** than **sigmoid** (the derivatives of its *exponential components* are more steep).

Activation Function: tanh

- Deciding between **sigmoid** and **tanh** will depend on gradient strength requirement of an application. Like the **sigmoid**, **tanh** also has the **missing slope** problem. **Figure5** shows the **tanh** activation function:



It looks very similar to sigmoid function; in fact, it is a **scaled sigmoid** function.

Figure 5

Activation Function: ReLU

- **Rectified Linear Unit (ReLU)** is the most used activation function for applications based on **CNN** (Convolutional NN).
- It is a simple condition and has advantages over the other functions.
- The function is defined by the following formula:
$$f(x) = \begin{cases} 0 & \text{when } (x < 0) \\ x & \text{when } (x \geq 0) \end{cases}$$
- $f(x) = \max(x, 0)$
- The range of output is between **0** and **infinity**.
- **ReLU** finds applications in **computer vision** and **speech recognition**.

Activation Function: ReLU

- **Figure6** shows a **ReLU** activation function:

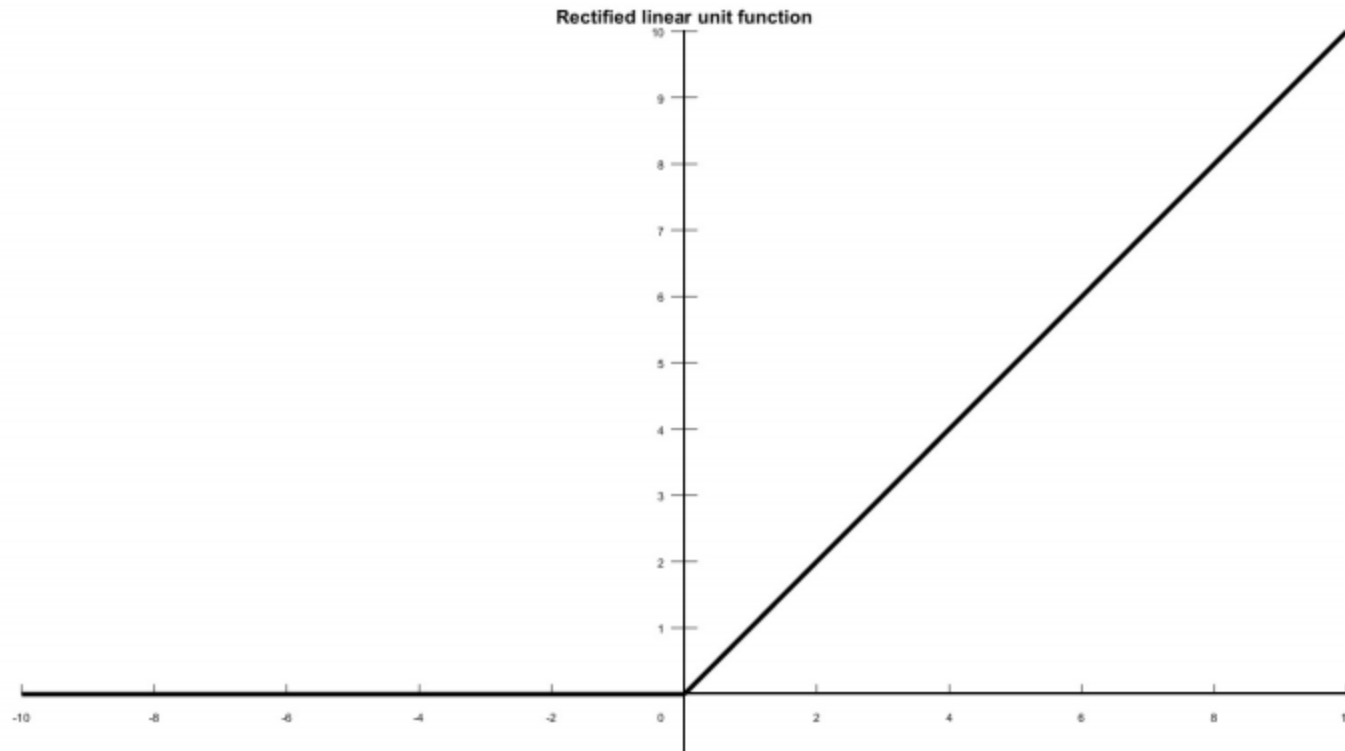


Figure 6

Which activation functions to use?

- The **sigmoid** is the most used activation function, but it suffers from the following setbacks:
 - Since it uses a logistic model, the computations are time-consuming and complex.
 - It causes gradients to vanish, and no signals pass through the neurons at some point.
 - It is slow in convergence.
 - It is not zero-centered.

Effects of Adjusting Weights

- Let's take a neuron node with only **one input** and **one output** (without a **bias** value) which is shown in **Figure 7**:

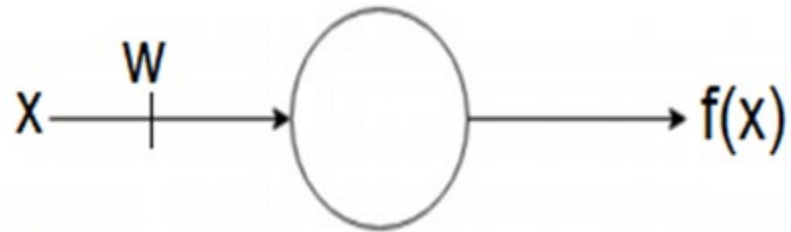


Figure 7: A single neuron node.

- The neuron's activation function, in this case, is the **sigmoid function**. What does changing **weight w** do in this simple network?
 - Figure 8** shows that changing the **weight** changes the slope of the output of the sigmoid activation function, which is helpful if we want to model different strengths of relationships between the **input** and **output** variables.

Effects of Adjusting Weights

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(-8, 8, 0.1)
w1 = 0.5
w2 = 1.0
w3 = 2.0
l1 = 'w1 = 0.5'
l2 = 'w2 = 1.0'
l3 = 'w3 = 2.0'
for w,l in [(w1, l1), (w2, l2), (w3, l3)]:
    f = 1/(1+np.exp(-x*w))
    plt.plot(x, f, label = l)
plt.xlabel('x')
plt.ylabel('y = f(x)')
plt.legend(loc = 2)
plt.show()
```

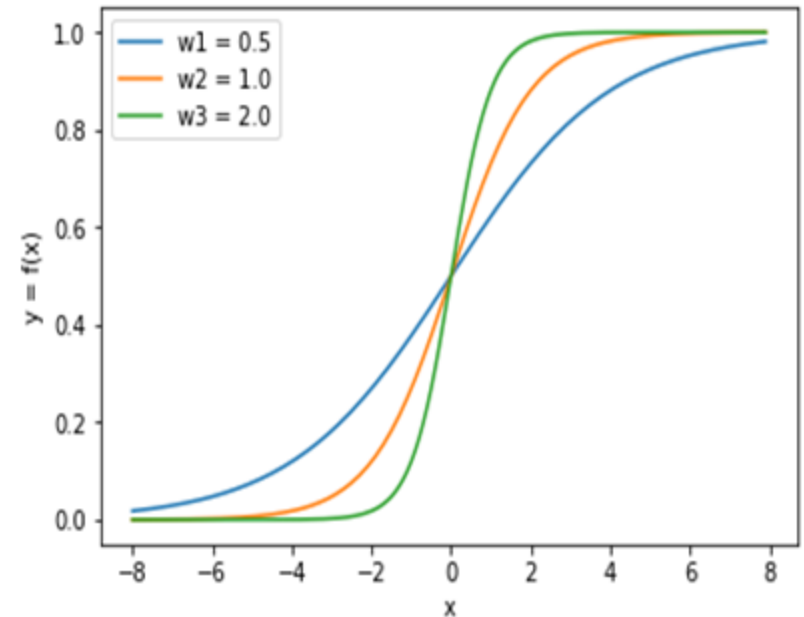


Figure 8: Effect of adjusting weights

Effects of Adjusting Bias

- The **weights** are real-valued numbers multiplied by the inputs and then summed up in the NN node.
 - In this case, the **w** has been increased to simulate a more defined “turn on” function.
- So, in other words, the **weighted input** to a neuron node with three inputs (**x_1** , **x_2** , and **x_3**) and their respective weights (**w_1** , **w_2** , and **w_3**) can be: **$x_1w_1 + x_2w_2 + x_3w_3 + b$**
 - where the **b** is the weight of the **+1** bias element of the neuron node, and
 - the inclusion of this **bias weight** enhances the flexibility of the neuron node.
 - The effect of **bias** adjustment is shown in **Figure 9**.

Effects of Adjusting Bias

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(-8, 8, 0.1)
w = 5.0
b1 = -8.0
b2 = 0.0
b3 = 8.0
l1 = 'b1 = -8.0'
l2 = 'b2 = 0.0'
l3 = 'b3 = 8.0'
for b, l in [(b1, l1), (b2, l2), (b3, l3)]:
    f = 1/(1 + np.exp(-(x*w) + b))
    plt.plot(x, f, label = l)
plt.xlabel('x')
plt.ylabel('y = f(x)')
plt.legend(loc = 2)
plt.show()
```

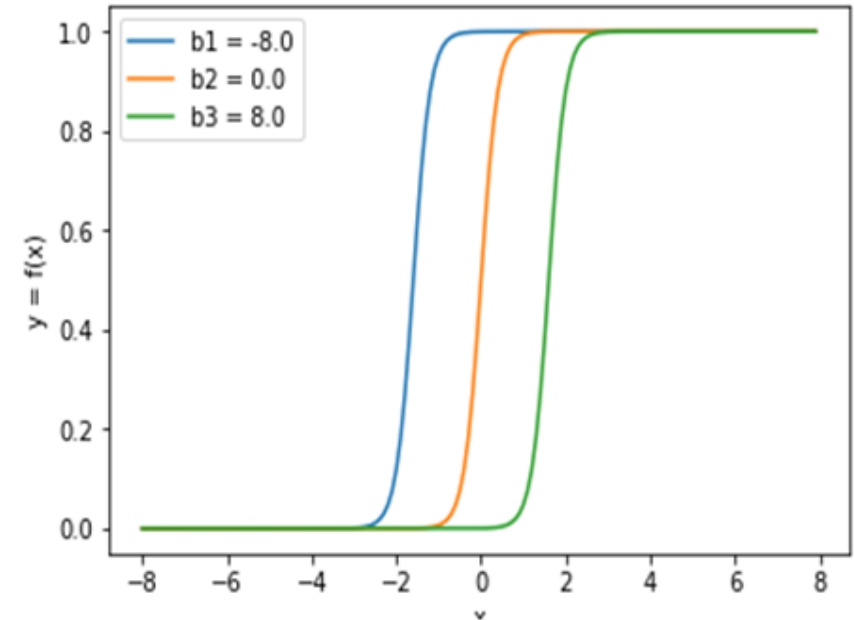


Figure 9: Effects of bias adjustments

Effects of Adjusting Bias

- **Figure 9** shows that by varying the **bias weight b** , you can change the **output** when the neuron node activates.
- Therefore, by adding a **bias** term, you can make a neuron node simulate a generic **if** function;
 - i.e. **if ($x > z$) then 1 else 0.**
 - Without a **bias** term, you cannot vary the **z** of the **if statement**; it will always be stuck around 0.
- This is very useful if you are trying to simulate conditional relationships.

What is a Gradient in a Neural network

- In a neural network, the **gradient** is a vector that represents the rate of change of the ***loss function*** with respect to each of the network's parameters (***weights*** and ***biases***).
- It essentially indicates the direction and magnitude of the change needed in the parameters to reduce the ***error*** (or ***loss***) of the network.

Gradient Descent Optimization

- **Gradient descent** is an iterative method to find the *minimum* of a function (*gradient descent will be very clear from the **perceptron learning** section*).
- **Gradient descent** starts with an initial set of **weights**; in each step, it **decreases** each **weight** in proportion to its partial derivative:

$$w_i := w_i + \eta \times \frac{\partial \text{Error}_E(\bar{w})}{\partial w_i}$$

- where η , the gradient descent step size, is called the **learning rate**.
- **The learning rate**, as well as the features and the data, is given as input to the learning algorithm.
- The partial derivative specifies **how much a small change in the weight would change the error**.

The Perceptron

- The **perceptron** is the simplest form of neural net with **one neuron**.
- It consists of single **neuron** with adjustable *weights* and a **hard limiter** output function (depends on the application)
- A **perceptron** with two input is shown in **Figure 10**:

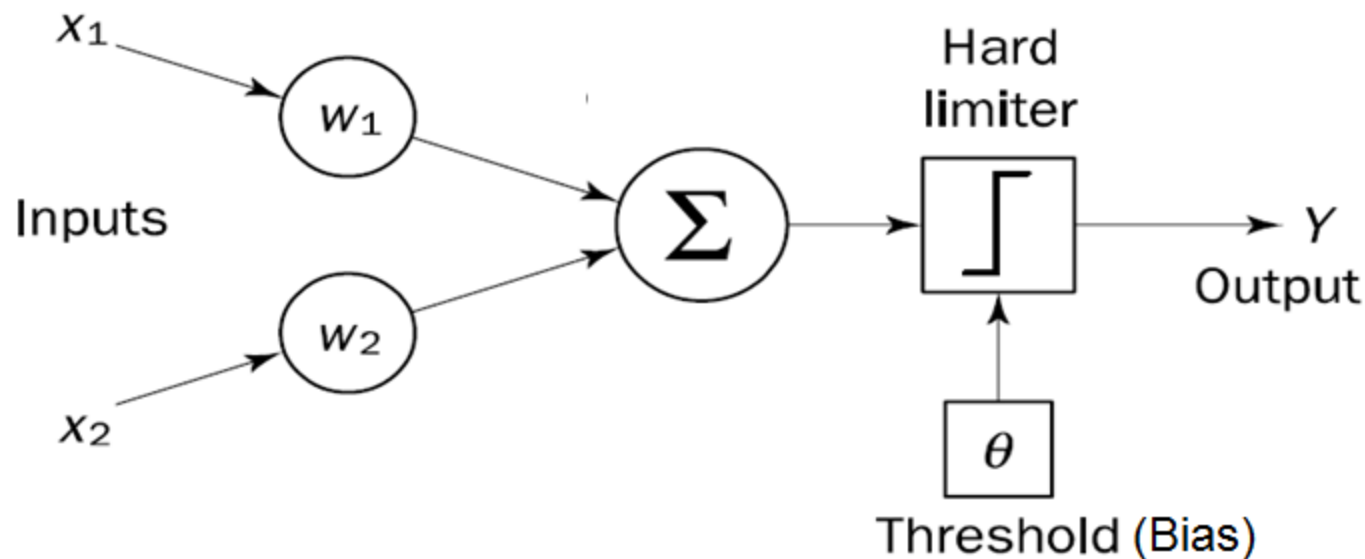
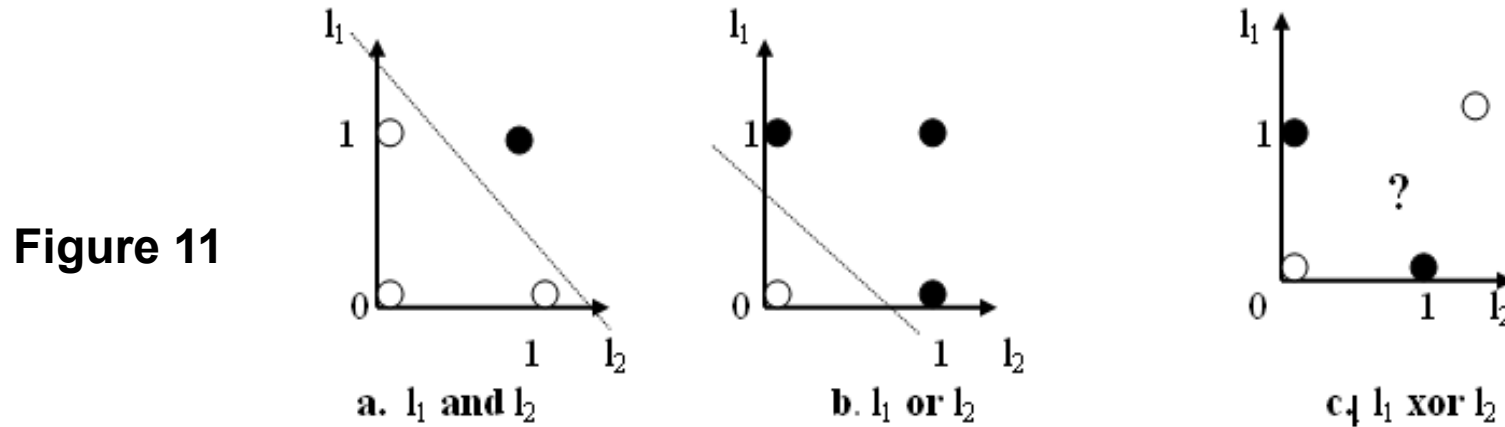


Figure 10: A perceptron with two input.

Linear Separability in Perceptrons

- **Figure 11** and **Figure 12** show three different Boolean functions of two inputs, the AND, OR, and XOR functions



- Each function is represented as a 2D plot, based on the values of the two inputs (**black dots** indicate **1**, and **white dots** indicate **0**)
- A perceptron can represent a function only if there is some line that separates all the **white dots** from the **black dots** called a **linearly separable** function.
- Thus, a perceptron can represent **AND**, and **OR**, but **not XOR** !

Linear Separability in Perceptrons

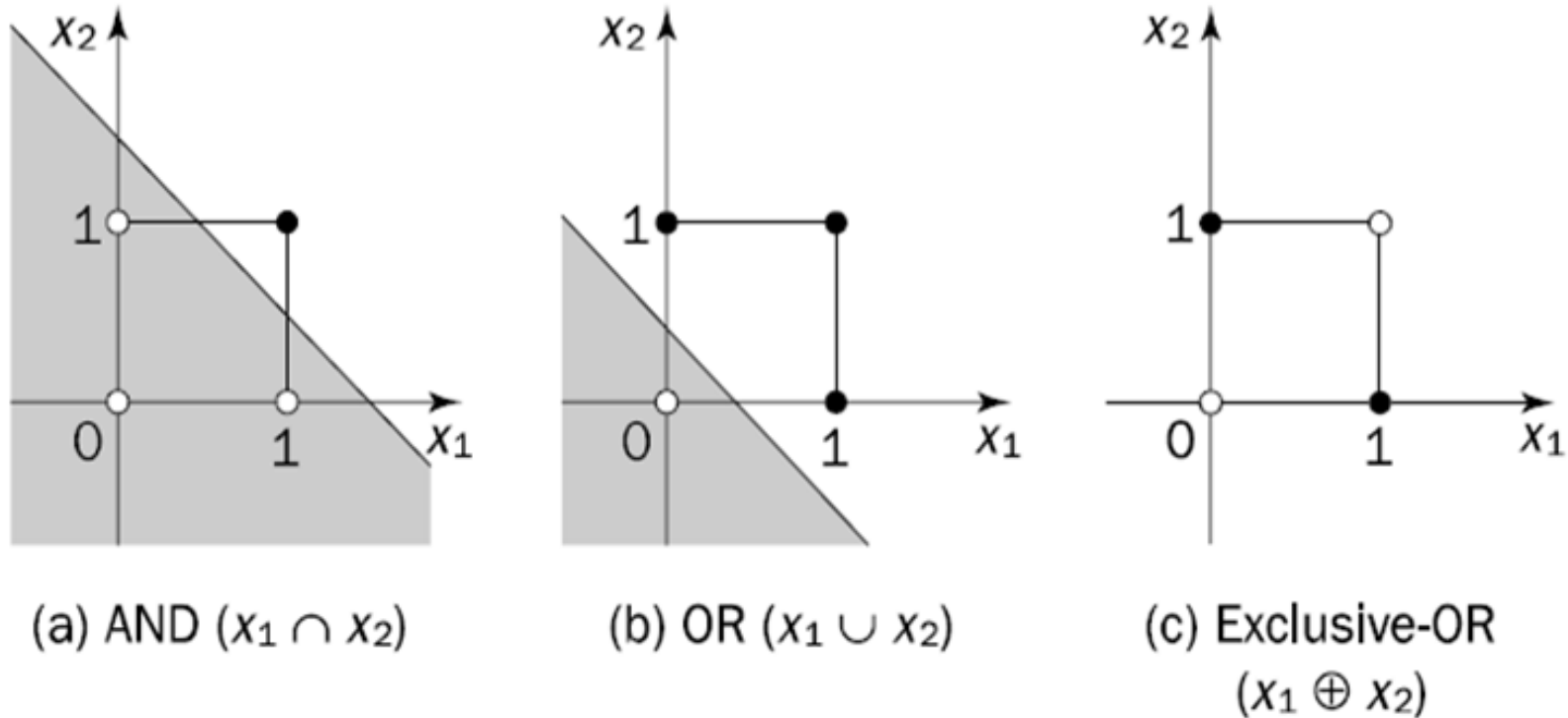


Figure 12: Two-dimensional plots of basic logical operations

The Perceptron Learning Rule (gradient free)

- Rosenblatt first proposed the **perceptron learning rule** in 1960, called the **delta rule**.
- Using the **delta rule**, we can derive the **perceptron training algorithm** for classification tasks
- This **delta rule** is **not based on gradient descent**, but it **works for linearly separable data**.
- This is done by making small adjustments in the **weights** to reduce the difference between the **perceptron's actual (calculated/predicted) and desired (given/expected) outputs**.
 - The **initial weights** are randomly assigned in the range $[-0.5, 0.5]$ and then updated to obtain the output consistent with the training examples

The Perceptron Learning Rule (gradient free)

- If at iteration $\mathbf{p}$, the **calculated/predicted** output is $\mathbf{Y}(\mathbf{p})$ and the **desired** output is $\mathbf{Y}_d(\mathbf{p})$, then the **error** $\mathbf{e}$ at iteration $\mathbf{p}$ is given by

The **weighted_sum** of the perceptron =
$$\sum_{i=1}^n (x_i w_i) \quad (1)$$

The **intermediate output**, $\mathbf{X} = \sum_{i=1}^n (x_i w_i) - b$ or $\sum_{i=1}^n (x_i w_i) + b \quad (2)$

The **calculated output** $\mathbf{Y}_d$ for a **binary (step)** output at iteration $\mathbf{p}$ is given as:

$$\mathbf{Y}_d(\mathbf{p}) = \begin{cases} 1 & \text{if } X \geq 0 \\ 0 & \text{if } X < 0 \end{cases}, \text{ where } \mathbf{X} \text{ is the } \textit{intermediate} \\ \textit{output} \text{ of the perceptron}$$

The Perceptron Learning Rule (gradient free)

- The **error** e at iteration p is given as:

$$e(p) = Y_d(p) - Y(p), \text{ where } p = 1, 2, 3, \dots \quad (3)$$

- If the error $e(p)$ is **positive**, we need to **increase** $Y(p)$
- If the error is **negative**, we need to **decrease** $Y(p)$
- Taking into account that each perceptron input contributes $x_i(p) \times w_i(p)$ with a **bias** (or **threshold**) θ to the total input $X(p)$
- Based on this, the **perceptron learning rule** is established.
- If the errors of an iteration are not zero, the weights of the inputs must be updated as per the given rule:

$$w_i(p+1) = w_i(p) + \alpha \times x_i(p) \times e(p), \quad (4)$$

where α is the **learning rate**, a positive constant **less than unity**.

The Perceptron Learning Rule (gradient free)

- The **equation (4)** can be modified as:

$$\mathbf{w}_i(\mathbf{p}+1) = \mathbf{w}_i(\mathbf{p}) + \Delta \mathbf{w}_i(\mathbf{p}), \quad (5)$$

where $\Delta \mathbf{w}_i(\mathbf{p}) = \alpha \times \mathbf{x}_i(\mathbf{p}) \times \mathbf{e}(\mathbf{p})$ (see equation (4))

- Where $\Delta \mathbf{w}_i(\mathbf{p})$ is the weight correction at iteration $\mathbf{p}$.
- The weight correction is computed by the **delta rule**.
- **Adam optimization** is not important for ***linear separable data***.
- Hence, the perceptron training for the two-input binary data is primarily based on *non-gradient iteration* (called the **delta rule**).

The Implementation of the Delta Rule

- Step1:Initialization

- Set initial weights $w_1, w_2, \dots, w_n$ and base θ to random numbers in the range $[-0.5, 0.5]$

- Step2: Activation

- Activate the perceptron by applying inputs $x_1(p), x_2(p), \dots, x_n(p)$ and **desired** output $Y_d(p)$. Calculate the **actual output** $Y(p)$ at $p = 1$:

$$Y(p) = \text{step} \left[\sum_{i=1}^n x_i(p) * w_i(p) - \theta \right] \quad (1)$$

- Where n is the number of the perceptron inputs and **step** is the **activation function**

The Implementation of the Delta Rule

- Step3: Weight training

- Update the weights of the perceptron

$$w_i(p+1) = w_i(p) + \Delta w_i(p) \quad (2)$$

- Where $\Delta w_i(p)$ is the weight correction at iteration p . The weight correction is computed by the **delta rule**:

$$\Delta w_i(p) = \alpha \times x_i(p) \times e(p) \quad (3)$$

- Step4: Iteration

- Increase iteration p by one, go back to step 2 and repeat the process until **convergence** (all the $y(p)$ focus with $y_d(p)$ without error)

Train Perceptron for AND and OR functions

- The truth tables for operations AND, OR and XOR are shown in **Table 1**. The perceptron must be trained to classify the input patterns

Table 1 Truth tables for the basic logical operations

Input variables		AND	OR	Exclusive-OR
x_1	x_2	$x_1 \cap x_2$	$x_1 \cup x_2$	$x_1 \oplus x_2$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

- The training process for AND function is shown in **Table 2**

Table 2 Example of perceptron learning: the logical operation AND

Epoch	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
1	0	0	0	0.3	-0.1	0	0	0.3	-0.1
	0	1	0	0.3	-0.1	0	0	0.3	-0.1
	1	0	0	0.3	-0.1	1	-1	0.2	-0.1
	1	1	1	0.2	-0.1	0	1	0.3	0.0
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	-1	0.2	0.0
	1	1	1	0.2	0.0	1	0	0.2	0.0
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	-1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	-1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

Threshold: $\theta = 0.2$; learning rate: $\alpha = 0.1$.

Why Can a Perceptron Learn Only Linearly Separable Functions?

- The fact that a perceptron can learn only **linearly separable functions** based on the following equation:

$$X = \sum_{i=1}^n x_i w_i$$
$$Y = \begin{cases} +1 & \text{if } X \geq \theta \\ -1 & \text{if } X < \theta \end{cases}$$

- The perceptron output Y is 1 only if the total weighted input X is greater than or equal to the threshold, θ . This means that the entire input space is divided in two along a boundary defined by $X = \theta$
- A separating line for the operation AND is defined by the equation $x_1 w_1 + x_2 w_2 = \theta$

Why Can a Perceptron Learn Only Linearly Separable Functions?

- If we substitute values for weights w_1 and w_2 and threshold θ given in **Table 2**, we obtain one of the possible separating lines as (see 5th iteration):

$$0.1x_1 + 0.1x_2 = 0.2 \quad \text{or} \quad x_1 + x_2 = 2$$

- Thus, the region below the boundary line, where the output is **0**, is given by $x_1 + x_2 - 2 < 0$,
- And the region above this line, where the output is **1**, is given by $x_1 + x_2 - 2 \geq 0$
- So a perceptron can learn only **linear separable functions** and there are not many such functions!

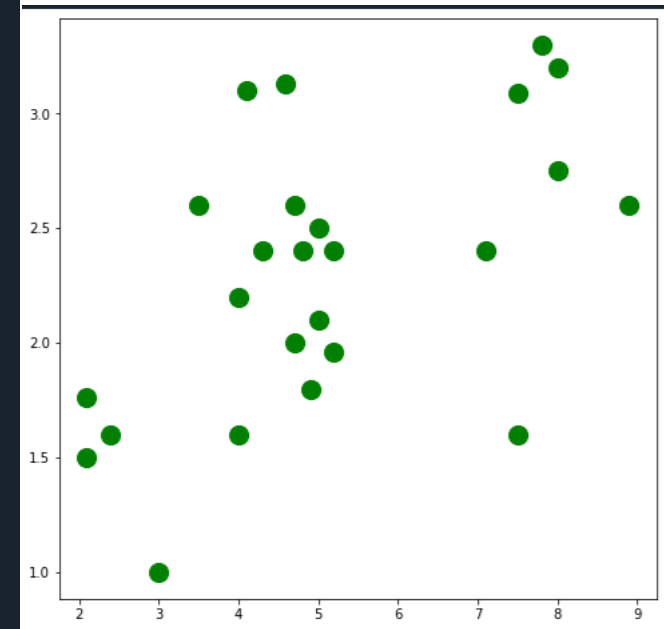
Linear Regression with Perceptron

<https://www.analyticsvidhya.com/blog/2021/11/a-comprehensive-guide-to-linear-regression-with-pytorch/>

```
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch.autograd import Variable
#Dataset is created using NumPy arrays.
x_train = np.array ([[4.7], [2.4], [7.5], [7.1], [4.3],
                    [7.8], [8.9], [5.2], [4.59], [2.1],
                    [8], [5], [7.5], [5], [4],
                    [8], [5.2], [4.9], [3], [4.7],
                    [4], [4.8], [3.5], [2.1], [4.1]],
                    dtype = np.float32)

y_train = np.array ([[2.6], [1.6], [3.09], [2.4], [2.4],
                    [3.3], [2.6], [1.96], [3.13], [1.76],
                    [3.2], [2.1], [1.6], [2.5], [2.2],
                    [2.75], [2.4], [1.8], [1], [2],
                    [1.6], [2.4], [2.6], [1.5], [3.1]],
                    dtype = np.float32)

#Visualizing the data.
plt.figure(figsize=(8,8))
plt.scatter(x_train, y_train, c='green', s=200, label='Original data')
plt.show()
```



Linear Regression with Perceptron

<https://www.analyticsvidhya.com/blog/2021/11/a-comprehensive-guide-to-linear-regression-with-pytorch/>

```
7 import numpy as np
8 import matplotlib.pyplot as plt
9 import torch
10 from torch.autograd import Variable
11 #Dataset is created using NumPy arrays.
12 x_train = np.array ([[4.7], [2.4], [7.5], [7.1], [4.3],
13                     [7.8], [8.9], [5.2], [4.59], [2.1],
14                     [8], [5], [7.5], [5], [4],
15                     [8], [5.2], [4.9], [3], [4.7],
16                     [4], [4.8], [3.5], [2.1], [4.1]],
17                     dtype = np.float32)
18
19 y_train = np.array ([[2.6], [1.6], [3.09], [2.4], [2.4],
20                     [3.3], [2.6], [1.96], [3.13], [1.76],
21                     [3.2], [2.1], [1.6], [2.5], [2.2],
22                     [2.75], [2.4], [1.8], [1], [2],
23                     [1.6], [2.4], [2.6], [1.5], [3.1]],
24                     dtype = np.float32)
25
26 #Visualizing the data.
27 plt.figure(figsize=(8,8))
28 plt.scatter(x_train, y_train, c='green', s=200, label='Original data')
29 plt.show()
30
31 X_train = torch.from_numpy(x_train) #Convert numpy arrays into Pytorch arrays
32 Y_train = torch.from_numpy(y_train) #Convert numpy arrays into Pytorch arrays
33 print('requires_grad for X_train: ', X_train.requires_grad)
34 print('requires_grad for Y_train: ', Y_train.requires_grad)
```

X_train	Y_train
tensor([[4.7000],	tensor([[2.6000],
[2.4000],	[1.6000],
[7.5000],	[3.0900],
[7.1000],	[2.4000],
[4.3000],	[2.4000],
[7.8000],	[3.3000],
[8.9000],	[2.6000],
[5.2000],	[1.9600],
[4.5900],	[3.1300],
[2.1000],	[1.7600],
[8.0000],	[3.2000],
[5.0000],	[2.1000],
[7.5000],	[1.6000],
[5.0000],	[2.5000],
[4.0000],	[2.2000],
[8.0000],	[2.7500],
[5.2000],	[2.4000],
[4.9000],	[1.8000],
[3.0000],	[1.0000],
[4.7000],	[2.0000],
[4.0000],	[1.6000],
[4.8000],	[2.4000],
[3.5000],	[2.6000],
[2.1000],	[1.5000],
[4.1000]])	[3.1000]])

Linear Regression with Perceptron

<https://www.analyticsvidhya.com/blog/2021/11/a-comprehensive-guide-to-linear-regression-with-pytorch/>

```
39 #Parameters of the model are W1 and b1, which is weight and bias respectively.
40 #Let us have a look at the parameters that are defined.
41 input_size = 1
42 hidden_size = 1
43 output_size = 1
44 learning_rate = 0.001
45 w1 = torch.rand(input_size,
46                 hidden_size,
47                 requires_grad=True)
48
49 b1 = torch.rand(hidden_size,
50                 output_size,
51                 requires_grad=True)
52
53 for iter in range(1, 5000):
54     y_pred = X_train.mm(w1).clamp(min=0).add(b1)
55     loss = (y_pred - Y_train).pow(2).sum()
56     if iter % 100 == 0:
57         print(iter, loss.item())
58     loss.backward()
59     with torch.no_grad():
60         w1 -= learning_rate * w1.grad
61         b1 -= learning_rate * b1.grad
62         w1.grad.zero_()
63         b1.grad.zero_()
64 #Let us check the optimized value for W1 and b1:
65 print(w1, b1)
```

2700	loss =	6.119896411895752
2800	loss =	6.119896411895752
2900	loss =	6.119896411895752
3000	loss =	6.119896411895752
3100	loss =	6.119896411895752
3200	loss =	6.119896411895752
3300	loss =	6.119896411895752
3400	loss =	6.119896411895752
3500	loss =	6.119896411895752
3600	loss =	6.119896411895752
3700	loss =	6.119896411895752
3800	loss =	6.119896411895752
3900	loss =	6.119896411895752
4000	loss =	6.119896411895752
4100	loss =	6.119896411895752
4200	loss =	6.119896411895752
4300	loss =	6.119896411895752
4400	loss =	6.119896411895752
4500	loss =	6.119896411895752
4600	loss =	6.119896411895752
4700	loss =	6.119896411895752
4800	loss =	6.119896411895752
4900	loss =	6.119896411895752

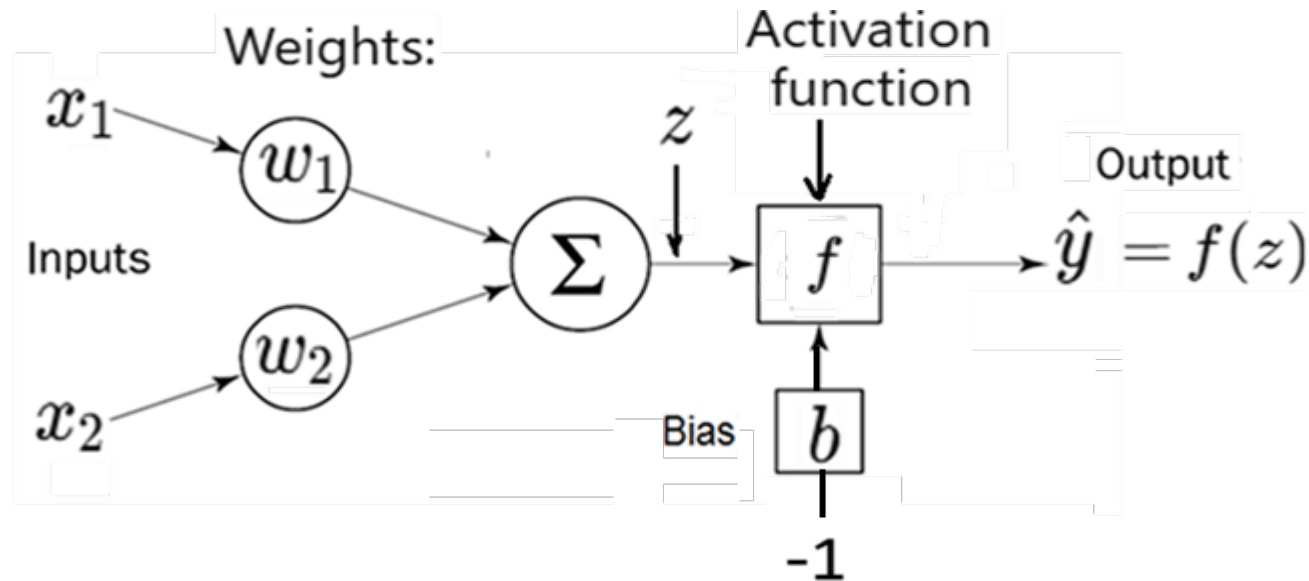
Linear Regression with Perceptron

<https://www.analyticsvidhya.com/blog/2021/11/a-comprehensive-guide-to-linear-regression-with-pytorch/>

```
68 #Let us check the optimized value for W1 and b1:
69 print ('w1: ', w1)
70 print ('b1: ', b1)
71 #Getting the prediction values using the weights in the linear equation.
72 getX = float(input("Enter x_tain value: "))
73 predicted_in_tensor = getX * w1 + b1
74 print ("The predicted output = ", Variable(predicted_in_tensor) )
```

```
w1:  tensor([[0.1751]], requires_grad=True)
b1:  tensor([[1.4045]], requires_grad=True)
Enter x_tain value: 2.4
The predicted output =  tensor([[1.8247]])
```

Using Perceptron for Non-linear Separable Data



- Inputs: x_1, x_2
- Weights: w_1, w_2
- Bias: b
- The linear combination (pre-activation) is:
$$z = w_1x_1 + w_2x_2 + b$$
- Sigmoid Activation function: f
- The output of the perceptron is: $\hat{y} = f(z)$

Using Perceptron for Non-linear Separable Data

- A sample of data is shown below:

x_1	x_2	y
0.3	0.4	0.6

- Learning rate, $\alpha = 0.1$

- Output of the perceptron, $y' = \text{sigmoid}(z) = \frac{1}{1+e^{-z}}$

- Error $e = y - y'$

- Calculate the **error gradient** (δ) of the perceptron:

$$\delta = y'(1 - y') \times e$$

- Calculate the **gradients** of w_1 , w_2 and b :

$$\nabla w_1 = \alpha \times x_1 \times \delta$$

$$\nabla w_2 = \alpha \times x_2 \times \delta$$

$$\nabla b = \alpha \times (-1) \times \delta$$

Using Perceptron for Non-linear Separable Data

- **Updated weights and bias:**

$$\mathbf{w}_{1\text{new}} = \mathbf{w}_1 + \nabla \mathbf{w}_1$$

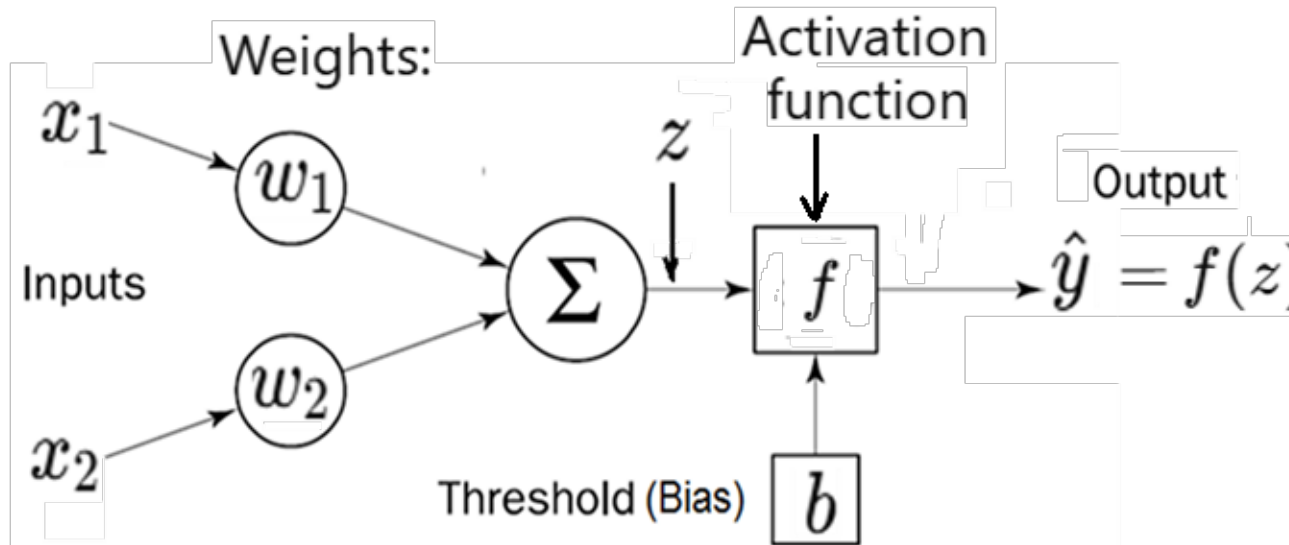
$$\mathbf{w}_{2\text{new}} = \mathbf{w}_2 + \nabla \mathbf{w}_2$$

$$\mathbf{b}_{\text{new}} = \mathbf{b} + \nabla \mathbf{b}$$

- **Perceptron programming Exercises:**

- Train a perceptron for getting 2-input OR and AND functions. Assume $\mathbf{w}_1 = 0.3$, $\mathbf{w}_2 = -0.1$, $\mathbf{b} = 0.2$ and $\alpha = 0.1$
- Consider a perceptron with two real-valued inputs and an output unit with a ***sigmoid activation*** function (for *non-linearly separable data*). All the initial weights and the bias are equal to **0.5**. Assume that the output ($\mathbf{y}$) should be **0.6** for the input $\mathbf{x}_1 = 0.3$ and $\mathbf{x}_2 = 0.4$. Show how the iterations on the given data supports the perceptron to reach the output (assume $\alpha = 0.1$)

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)



- Inputs: x_1, x_2
- Weights: w_1, w_2
- Bias: b
- The **linear combination (pre-activation)** is:
$$z = w_1x_1 + w_2x_2 + b$$
- Activation function: f
- The **output** of the perceptron is: $\hat{y} = f(z)$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- **Goal**: Use Adam to update ***weights*** and ***bias***.
- **Input data and desired output** : $x_1 = 1.0$, $x_2 = 0$, $y = 1$ (desired o/p)
- **Initial values of weights and bias**: $w_1 = 0.5$, $w_2 = -0.5$, $b = 0$
- **Learning rate**: $\alpha = 0.01$
- **Initialize Adam Hyper Parameters**:
 - First ***moment***: $m_0 = 0$
 - Second ***moment***: $v_0 = 0$
 - ***Time step***: $t = 1$ (first value calculation)
 - ***Exponential decay rates***: $\beta_1 = 0.9$, $\beta_2 = 0.999$
 - ***Numerical stability*** constant: $\epsilon = 10^{-8}$
- **Loss**: $L = \frac{1}{2}(\hat{y} - y)^2 = (\hat{y} - y)$
- **Gradient g of loss**: $\nabla_{\theta} L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b} \right]$ where $\theta \in (w_1, w_2, b)$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- **Forward pass:**

- **The sum-of-weighted-inputs or pre-activation (z):**

$$\begin{aligned} z &= w_1x_1 + w_2x_2 + b \\ &= (0.5 \times 1.0) + (-0.5 \times 0) + 0 = \mathbf{0.5} \end{aligned}$$

- **Perceptron output ($\hat{y}$):**

$$\begin{aligned} \hat{y} &= f(z) = \text{sigmoid}(z) = \frac{1}{1+e^{-z}} \\ &= 1/(1 + 2.7183^{-0.5}) = \mathbf{0.6225} \end{aligned}$$

- **The derivative of the perceptron output:**

$$\hat{y}' = f'(z) = \hat{y} (1 - \hat{y}) = 0.6225(1 - 0.6225) = \mathbf{0.235}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Compute gradient g of Loss (for w_1, w_2, b):

- $\delta (\text{loss-parameter}) = (\hat{y} - y) \times f'(z)$

$$\delta = (0.6225 - 1) \times 0.235 = \mathbf{-0.089}$$

- $\nabla_{w_1} L = \frac{\partial L}{\partial w_1} = \delta \mathbf{x}_1$

$$= -0.089 \times 1.0 = \mathbf{-0.089} \text{ (gradient of } \mathbf{w}_1 \text{ with respect to } \mathbf{x}_1 \text{ and } \delta \text{)}$$

- $\nabla_{w_2} L = \frac{\partial L}{\partial w_2} = \delta \mathbf{x}_2$

$$= -0.089 \times 0 = \mathbf{0} \text{ (gradient of } \mathbf{w}_2 \text{ with respect to } \mathbf{x}_2 \text{ and } \delta \text{)}$$

- $\nabla_b L = \frac{\partial L}{\partial b} = \delta$

$$= \mathbf{-0.089} \text{ (gradient of } \mathbf{b} \text{ with respect to } \delta \text{)}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Apply Adam to update moment terms based on $\theta(w_1, w_2, \text{ and } b)$:
- Update the first moment m_t (at $t=1$) for w_1 :

$$m_{t_{w1}} = \beta_1 m_{(t-1)} + (1 - \beta_1) g_{t(w1)}$$

$$m_{1w1} = \beta_1 m_0 + (1 - \beta_1) \times \frac{\partial L}{\partial w_1}$$

$$= \beta_1 m_0 + (1 - \beta_1) \times \delta x_1$$

- Update the 2nd moment v_t (at $t = 1$) for w_1 :

$$v_{t_{w1}} = \beta_2 v_{(t-1)} + (1 - \beta_2)(g_{t(w1)})^2$$

$$v_{1w1} = \beta_2 v_0 + (1 - \beta_2) \times \left(\frac{\partial L}{\partial w_1}\right)^2$$

$$= \beta_2 v_0 + (1 - \beta_2) \times (\delta x_1)^2$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Update the first moment m_t (at $t = 1$) for gradient loss due to w_2

$$m_{t_{w_2}} = \beta_1 m_{(t-1)} + (1 - \beta_1) g_{t(w_2)}$$

$$m_{1w_2} = \beta_1 m_0 + (1 - \beta_1) \times \frac{\partial L}{\partial w_2}$$

$$= \beta_1 m_0 + (1 - \beta_1) \times \delta x_2$$

- Update the 2nd moment v_t (at $t = 1$) for gradient loss due to w_2

$$v_{t_{w_2}} = \beta_2 v_{(t-1)} + (1 - \beta_2) (g_{t(w_2)})^2$$

$$v_{1w_2} = \beta_2 v_0 + (1 - \beta_2) \times \left(\frac{\partial L}{\partial w_2} \right)^2$$

$$= \beta_2 v_0 + (1 - \beta_2) \times (\delta x_2)^2$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Update the first moment m_t (at $t = 1$) for gradient loss due to b :

$$m_{t_b} = \beta_1 m_{(t-1)} + (1 - \beta_1) g_{t(b)}$$

$$m_{1b} = \beta_1 m_0 + (1 - \beta_1) \times \frac{\partial L}{\partial b}$$

$$= \beta_1 m_0 + (1 - \beta_1) \times \delta$$

- Update the 2nd moment v_t (at $t = 1$) for gradient loss due to b :

$$v_{t_b} = \beta_2 v_{(t-1)} + (1 - \beta_2) (g_{t(b)})^2$$

$$v_{1b} = \beta_2 v_0 + (1 - \beta_2) \times \left(\frac{\partial L}{\partial b}\right)^2$$

$$= \beta_2 v_0 + (1 - \beta_2) \times (\delta)^2$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Bias correction of the 1st moment m_t (at $t = 1$) for gradient loss due to w_1 :

$$\hat{m}_{tw1} = \frac{m_{tw1}}{1 - \beta_1^t}$$

$$\hat{m}_{1w1} = \frac{m_{1w1}}{1 - \beta_1^1}$$

- Bias correction of the 2nd moment v_t (at $t = 1$) for gradient loss due to w_1 :

$$\hat{v}_{tw1} = \frac{v_{tw1}}{1 - \beta_2^t}$$

$$\hat{v}_{1w1} = \frac{v_{1w1}}{1 - \beta_2^1}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- **Bias correction of the 1st moment m_t (at $t = 1$) for gradient loss due to w_2 :**

$$\hat{m}_{tw_2} = \frac{m_{tw_2}}{1 - \beta_1^t}$$

$$\hat{m}_{1w_2} = \frac{m_{1w_2}}{1 - \beta_1^1}$$

- **Bias correction of the 2nd moment v_t (at $t = 1$) for gradient loss due to w_2 :**

$$\hat{v}_{tw_2} = \frac{v_{tw_2}}{1 - \beta_2^t}$$

$$\hat{v}_{1w_2} = \frac{v_{1w_2}}{1 - \beta_2^1}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- **Bias correction of the 1st moment m_t (at $t = 1$) for gradient loss due to b :**

$$\hat{m}_{tb} = \frac{m_{tb}}{1 - \beta_1^t}$$

$$\hat{m}_{1b} = \frac{m_{1b}}{1 - \beta_1^1}$$

- **Bias correction of the 2nd moment v_t (at $t = 1$) for gradient loss due to b :**

$$\hat{v}_{tb} = \frac{v_{tb}}{1 - \beta_2^t}$$

$$\hat{v}_{1b} = \frac{v_{1b}}{1 - \beta_2^1}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Update parameter $\theta \in (w_1, w_2, b)$

$$\theta_t = \theta_{t-1} - \alpha \times \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

- Update parameter $\theta = w_1 (t = 1)$:

$$w_{1\text{new}} = w_{1\text{old}} - \alpha \times \frac{\widehat{m}_{1w1}}{\sqrt{\widehat{v}_{1w1}} + \epsilon}$$

- Update parameter $\theta = w_2 (t = 1)$:

$$w_{2\text{new}} = w_{2\text{old}} - \alpha \times \frac{\widehat{m}_{1w2}}{\sqrt{\widehat{v}_{1w2}} + \epsilon}$$

- Update parameter $\theta = b (t = 1)$:

$$b_{\text{new}} = b_{\text{old}} - \alpha \times \frac{\widehat{m}_{1b}}{\sqrt{\widehat{v}_{1b}} + \epsilon}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Update Adam moment for w_1 at $t = 1$:
- Update the 1st moment m_t (at $t = 1$) for gradient loss due to w_1

$$m_{tw1} = \beta_1 m_{t-1} + (1 - \beta_1)g_{t(w1)}$$

$$m_{1w1} = \beta_1 m_0 + (1 - \beta_1) \times \frac{\partial L}{\partial w_1} = \beta_1 m_0 + (1 - \beta_1) \times \delta x_1$$

$$= (0.9 \times 0) + (1 - 0.9) \times -0.089 = 0.1 \times -0.089 = \mathbf{-0.0089}$$

- Update the 2nd moment v_t (at $t = 1$) for gradient loss due to w_1

$$v_{tw1} = \beta_2 v_{t-1} + (1 - \beta_2)(g_{t(w1)})^2$$

$$v_{1w1} = \beta_2 v_0 + (1 - \beta_2) \times \left(\frac{\partial L}{\partial w_1}\right)^2 = \beta_2 v_0 + (1 - \beta_2) \times (\delta x_1)^2$$

$$= (0.999 \times 0) + (1 - 0.999) \times (-0.089)^2 = 0.001 \times 0.007921$$

$$= \mathbf{0.000007921}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Bias correction of the 1st moment m_t (at $t = 1$) for w_1

$$\widehat{m}_{tw1} = \frac{m_{tw1}}{1 - \beta_1^t}$$

$$\widehat{m}_{1w1} = \frac{m_{1w1}}{1 - \beta_1^1} = -0.0089 / (1 - 0.9) = -0.0089 / 0.1 = \mathbf{-0.089}$$

- Bias correction of the 2nd moment v_t (at $t = 1$) for w_1

$$\widehat{v}_{tw1} = \frac{v_{tw1}}{1 - \beta_2^t}$$

$$\widehat{v}_{1w1} = \frac{v_{1w1}}{1 - \beta_2^1} = 0.000007921 / (1 - 0.999) = 0.000007921 / 0.001 \\ = \mathbf{0.007921}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Adam update w_1 (at $t = 1$):

$$\begin{aligned}w_{1\text{new}} &= w_{1\text{old}} - \alpha \times \frac{\widehat{m}_{1w1}}{\sqrt{\widehat{v}_{1w1}} + \epsilon} \\&= 0.5 - (0.01 \times -0.089/(\sqrt{0.007921} + 10^{-8})) \\&= 0.5 - (0.01 \times (-0.089/0.089000001)) \\&= 0.5 - (0.01 \times (-0.999999988)) \\&= 0.5 + 0.009999999 = \mathbf{0.509999}\end{aligned}$$

The new $w_1 = 0.509999$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Update Adam hyperparameters for w_2 at $t = 1$:
- Update the first moment m_t (at $t = 1$) for gradient loss due to w_2

$$m_{t_{w2}} = \beta_1 m_{(t-1)} + (1 - \beta_1) g_{t(w2)}$$

$$m_{1w2} = \beta_1 m_0 + (1 - \beta_1) \times \frac{\partial L}{\partial w_2}$$

$$= \beta_1 m_0 + (1 - \beta_1) \times \delta x_2$$

$$= (0.9 \times 0) + (1 - 0.9) \times 0 = 0.1 \times 0 = 0$$

- Update the 2nd moment v_t (at $t = 1$) for gradient loss due to w_2

$$v_{t_{w2}} = \beta_2 v_{(t-1)} + (1 - \beta_2) (g_{t(w2)})^2$$

$$v_{1w2} = \beta_2 v_0 + (1 - \beta_2) \times \left(\frac{\partial L}{\partial w_2}\right)^2 = \beta_2 v_0 + (1 - \beta_2) \times (\delta x_2)^2$$

$$= (0.999 \times 0) + (1 - 0.999) \times (0)^2 = 0$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- **Bias correction of the 1st moment m_t (at $t = 1$) for gradient loss due to w_2**

$$\widehat{m}_{tw2} = \frac{m_{tw2}}{1 - \beta_1^t}$$

$$\widehat{m}_{1w2} = \frac{m_{1w2}}{1 - \beta_1^1} = 0 / (1 - 0.9) = 0 / 0.1 = 0$$

- **Bias correction of the 2nd moment v_t (at $t = 1$) for gradient loss due to w_2**

$$\widehat{v}_{tw2} = \frac{v_{tw2}}{1 - \beta_2^t}$$

$$\widehat{v}_{1w2} = \frac{v_{1w2}}{1 - \beta_2^1} = 0 / (1 - 0.999) = 0 / 0.001 = 0$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Adam update w_2 (at $t = 1$):

$$\begin{aligned}\text{New } w_2 &= w_{2\text{old}} - \alpha \times \frac{\widehat{m}_{1w2}}{\sqrt{\widehat{v}_{1w2}} + \epsilon} \\ &= -0.5 - (0.01 \times 0/(\sqrt{0} + 10^{-8})) \\ &= 0.5 - (0.01 \times 0) \\ &= 0.5 - 0 \\ &= 0.5\end{aligned}$$

The new $w_2 = 0.5$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- Update the first moment m_t (at $t = 1$) for gradient loss due to b

$$m_{tb} = \beta_1 m_{t-1} + (1 - \beta_1) g_{t(b)}$$

$$m_{1b} = \beta_1 m_0 + (1 - \beta_1) \times \frac{\partial L}{\partial b}$$

$$= \beta_1 m_0 + (1 - \beta_1) \times \delta$$

$$= (0.9 \times 0) + (1 - 0.9) \times -0.089 = 0.1 \times -0.089 = \mathbf{-0.0089}$$

- Update the 2nd moment v_t (at $t = 1$) for gradient loss due to b

$$v_{tb} = \beta_2 v_{t-1} + (1 - \beta_2) (g_{t(b)})^2$$

$$v_{1b} = \beta_2 v_0 + (1 - \beta_2) \times \left(\frac{\partial L}{\partial b}\right)^2$$

$$= \beta_2 v_0 + (1 - \beta_2) \times (\delta)^2$$

$$= ((0.999 \times 0) + (1 - 0.999) \times (-0.089)^2 = 0.001 \times 0.007921$$

$$= \mathbf{0.000007921}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- **Adam update for b at $t = 1$:**
- **Bias correction of the 1st moment m_t (at $t = 1$) for gradient loss due to b :**

$$\hat{m}_{tb} = \frac{m_{tb}}{1 - \beta_1^t}$$

$$\hat{m}_{1b} = \frac{m_{1b}}{1 - \beta_1^1} = -0.0089 / (1 - 0.9) = -0.0089 / 0.1 = \mathbf{-0.089}$$

- **Bias correction of the 2nd moment v_t (at $t = 1$) for gradient loss due to b :**

$$\hat{v}_{tb} = \frac{v_{tb}}{1 - \beta_2^t}$$

$$\hat{v}_{1b} = \frac{v_{1b}}{1 - \beta_2^1} = 0.000007921 / (1 - 0.999) = 0.000007921 / 0.001$$
$$\mathbf{= 0.007921}$$

Adam Optimizer on a 2-I/P Perceptron(ChatGPT)

- **Adam Update b , at timestep $t = 1$:**

$$\begin{aligned}\text{New } b &= b_{\text{old}} - \alpha \times \frac{\widehat{m}_{1b}}{\sqrt{\widehat{v}_{1b}} + \epsilon} \\ &= 0 - (0.01 \times -0.089 / (\sqrt{0.007921} + 10^{-8})) \\ &= 0 - (0.01 \times (-0.089/0.089000001)) \\ &= 0 - (0.01 \times -0.99999999) \\ &= 0 - (-0.01) = \mathbf{0.009999}\end{aligned}$$

The new $b = 0.009999$

- **Next:**
- Calculate the output of the perceptron with the new values of w_1 , w_2 and b (next timesteps t) until to reach **Loss(L) = $(\hat{y} - y) = 0$**

Structure of a Multilayer ANN

- The structure of an **Multilayer ANN** is shown in **Figure 13**:

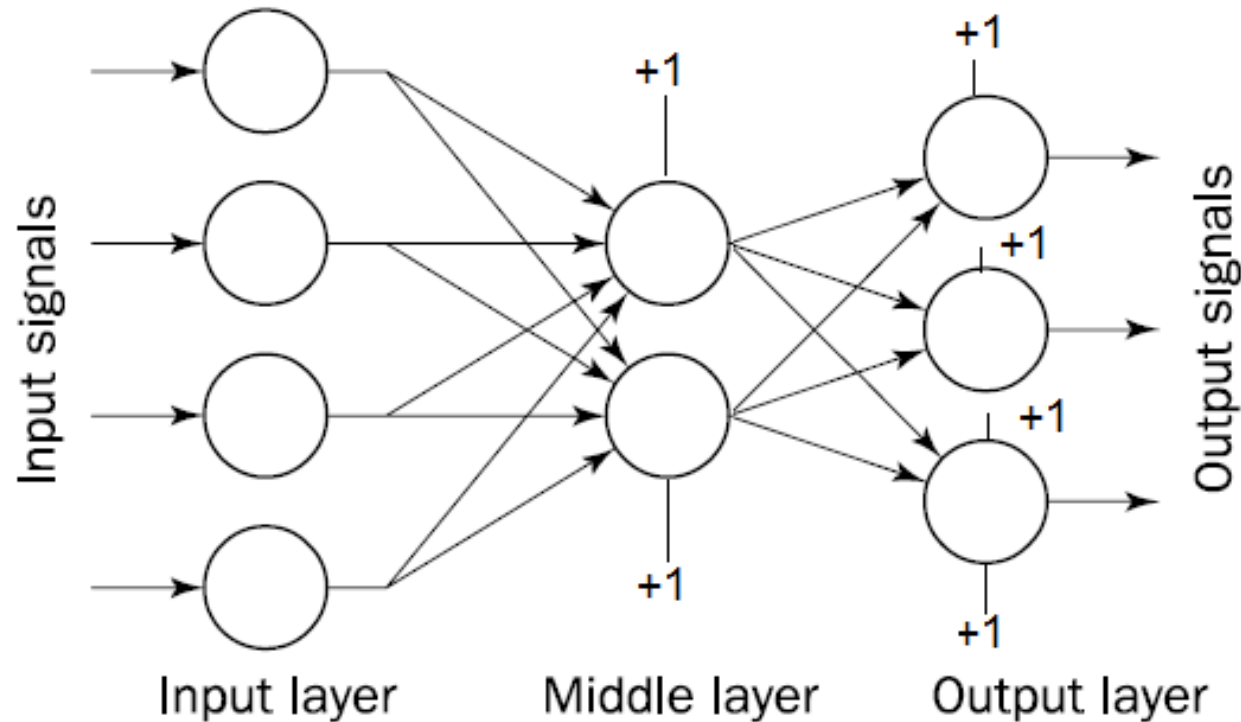


Figure 13: The structure of an ANN